```
-- Esquema del bingo (cartones numerados). Idempotente: se puede correr más de un
-- Usar una base de datos NUEVA para el bingo: la tabla "numeros" son los cartone
-- Reglas de integridad que sostiene la base (no el código de la web):
--   * cada cartón (0001-0500) es una fila única: no se puede vender dos veces
--   * un pago de Mercado Pago se guarda una sola vez (mp_payment_id es clave)
--   * una orden tiene como máximo un pago "asignado" (índice parcial)

create table if not exists ordenes (
  id                 uuid primary key,
  ref                text not null unique,            -- external_reference que v
  comprador_nombre   text not null,
  comprador_whatsapp text not null,
  cantidad           int  not null check (cantidad between 1 and 20),
  modo               text not null check (modo in ('azar', 'elegir')),
  numeros_elegidos   smallint[],
  monto              int  not null check (monto > 0), -- en pesos
  estado             text not null default 'pendiente'
                     check (estado in ('pendiente', 'pagada', 'vencida', 'requier
  expira_en          timestamptz not null,
  mp_preference_id   text,
  creada_en          timestamptz not null default now(),
  check ((modo = 'elegir') = (numeros_elegidos is not null and cardinality(numero
);

-- Entradas anticipadas de la compra (cada una incluye 1 cartón). cantidad = entr
alter table ordenes add column if not exists entradas int not null default 0 chec

-- Cortesías (invitaciones sin pago) y ajustes de cantidades hechos a mano desde
--   cortesia       : compra regalada, sin pago de Mercado Pago (monto 0)
--   monto_original : lo que se cobró antes del primer ajuste (para saber cuánto
--   ajustada_en    : cuándo se ajustaron las cantidades por última vez
alter table ordenes add column if not exists cortesia boolean not null default fa
alter table ordenes add column if not exists monto_original int;
alter table ordenes add column if not exists ajustada_en timestamptz;
do $$
begin
  -- El monto tiene que ser positivo, salvo en las cortesías (que no se cobran).
  if exists (select 1 from pg_constraint where conrelid = 'ordenes'::regclass and
    alter table ordenes drop constraint ordenes_monto_check;
  end if;
  if not exists (select 1 from pg_constraint where conrelid = 'ordenes'::regclass
    alter table ordenes add constraint ordenes_monto_valido check (monto > 0 or c
  end if;
end $$;
```

```
-- pagada_en: cuándo se acreditó el pago (para ordenar el registro de compras del
alter table ordenes add column if not exists pagada_en timestamptz;

-- Cancelar compras desde el panel. Una compra cancelada libera sus números (se g
-- numeros_anulados como registro) y nunca vuelve a recibirlos aunque llegue un p
alter table ordenes add column if not exists cancelada_en timestamptz;
alter table ordenes add column if not exists numeros_anulados smallint[];
do $$
declare c record;
begin
  -- Bases creadas antes: la regla de estados no incluía 'cancelada'.
  for c in select conname from pg_constraint
           where conrelid = 'ordenes'::regclass and contype = 'c'
             and pg_get_constraintdef(oid) like '%requiere_reembolso%'
             and pg_get_constraintdef(oid) not like '%cancelada%'
  loop
    execute format('alter table ordenes drop constraint %I', c.conname);
  end loop;
  if not exists (select 1 from pg_constraint where conrelid = 'ordenes'::regclass
                 and pg_get_constraintdef(oid) like '%cancelada%') then
    alter table ordenes add constraint ordenes_estado_valido
      check (estado in ('pendiente', 'pagada', 'vencida', 'requiere_reembolso', '
  end if;
end $$;

create table if not exists numeros (
  numero          smallint primary key check (numero between 1 and 9999),
  estado          text not null default 'libre' check (estado in ('libre', 'reser
  orden_id        uuid references ordenes (id),
  reservado_hasta timestamptz,
  check ((estado = 'libre') = (orden_id is null))
);

insert into numeros (numero)
select g from generate_series(1, 500) g
on conflict do nothing;

create table if not exists pagos (
  mp_payment_id      text primary key,
  orden_id           uuid references ordenes (id),
  external_reference text,
  estado_mp          text not null,
  monto              int,
  resultado          text not null default 'pendiente'
                     check (resultado in ('pendiente', 'asignado', 'requiere_reem
                                          'sin_orden', 'monto_distinto', 'no_apro
```

```
  detalle            text,
  creado_en          timestamptz not null default now(),
  actualizado_en     timestamptz not null default now()
);

create unique index if not exists un_pago_asignado_por_orden
  on pagos (orden_id) where resultado = 'asignado';

create table if not exists notificaciones (
  id        bigserial primary key,
  orden_id  uuid references ordenes (id),
  tipo      text not null,
  payload   jsonb not null,
  creada_en timestamptz not null default now()
);
```